

CS 6814: Probabilistic Proofs

Scribe Notes

September 1–24, 2026

Contents

September 1, 2026	2
September 3, 2026	6
September 8, 2026	11
September 10, 2026	15
September 15, 2026	19
September 17, 2026	27
September 22, 2026	30
September 24, 2026	35
References	38

September 1, 2026

Scribe Notes

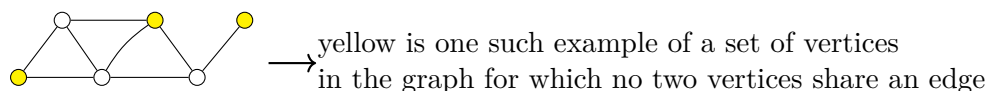
Day 03

Week 1 Recap

A *proof system* is a method of convincing a polynomial-time verifier of the truth of some statement.

Examples of statements:

- $\sqrt{2}$ is irrational.
- These two Ping-Pong balls are different colors (to a color-blind person).
- $G_0 \not\cong G_1$ (graph non-isomorphism).
- G has an independent set of size at least 3 (an NP-complete problem).



Complexity Classes and Types of Proof Systems

- NP: deterministic proof systems,

$$\text{NP} = \{L : L \text{ has a deterministic proof system}\}.$$

Examples include

$$\text{IND-SET} = \{(G, k) : G \text{ has an independent set of size } \geq k\}$$

and GRAPH ISOMORPHISM.

- MA: randomized proof systems.
- Interactive proofs: $\text{MA}(k)$ and $\text{IP}(k)$.
- Later: SZK (statistical zero knowledge), MIP, PCP, and IOP.

Studying and Comparing Proof Systems

- **Complexity-theoretic power:**

$$\text{NP} \subseteq \text{MA} \subseteq \text{AM}, \quad \text{coNP} \subseteq \text{IP}, \quad \text{IP} \subseteq \text{PSPACE}, \quad \text{GNI} \in \text{IP}, \quad \text{GNI} \notin \text{NP} \text{ (believed)}.$$

Polynomial space is the class of problems computable by a Turing machine using polynomial space.

- **Efficiency.**
- **Cryptographic properties:**
 - privacy: does the proof reveal anything about how the statement was proved?
 - transferability: if I prove something to you, can you prove it to somebody else?
- Connections to complexity theory, cryptography, and quantum computing.

Why Should You Care?

1. These objects are cool and give new ways to think about computation.
2. They connect to complexity theory:
 - NP-completeness;
 - because $\text{GNI} \in \text{AM}$, GI is probably not NP-complete;
 - hardness of approximation. For example, if there is an efficient algorithm A and a constant $\rho > 0$ such that whenever G has an independent set of size k , $A(G)$ returns one of size ρk , then $P = NP$.
 - different proof systems yield different insights.
3. Cryptography and computer safety:
 - probabilistic proof systems $\Rightarrow$ cryptographic proof systems;
 - privacy-preserving authentication (showing access without revealing private information);
 - private, scalable cryptocurrencies;
 - secure distributed computation (even when some machines may be malicious and the system is resilient against bugs);
 - and much more.

Goal (week): What Is the Power of IP?

Answer: exactly the languages computable by a Turing machine in polynomial space:

$$\text{IP} = \text{PSPACE}.$$

Goal for Today: Show $\text{coNP} \subseteq \text{IP}$

Construct an interactive proof for UNSAT, the language of unsatisfiable 3-CNF formulas:

$$\text{UNSAT} = \{\Phi \text{ a 3-CNF} : \forall x_1, \dots, x_n \in \{0, 1\}, \Phi(x_1, \dots, x_n) = 0\}.$$

Since 3-SAT is NP-complete, UNSAT is coNP-complete.

A formula in conjunctive normal form has the form

$$\Phi = \bigwedge_{i=1}^m (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3}),$$

where every ℓ_{ij} is a literal, namely x_v or $\overline{x_v}$ for some $v \in [n]$. For example,

$$\Phi = \underbrace{(x_1 \vee \overline{x_2} \vee x_3)}_{\text{clause 1}} \wedge \underbrace{(\overline{x_2} \vee x_3 \vee \overline{x_4})}_{\text{clause 2}}.$$

Designing a Proof System for a Related Problem

Suppose an algorithm computes a function

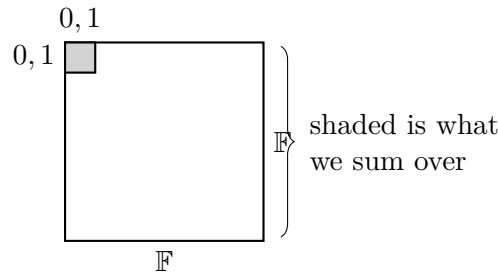
$$f : \mathbb{F}^n \rightarrow \mathbb{F},$$

where $\mathbb{F}$ is a finite field and $f \in \mathbb{F}^{\leq d}[x_1, \dots, x_n]$ has total degree d . The algorithm is given oracle access to the table

$$\{f(a_1, \dots, a_n) : a_i \in \mathbb{F}\}.$$

We want to verify whether, for a claimed $g \in \mathbb{F}$,

$$\sum_{b_1, \dots, b_n \in \{0,1\}} f(b_1, \dots, b_n) = g \in \mathbb{F}.$$



Without help, this takes time at least 2^n . Today: an interactive proof for this problem whose verifier runs in time $\text{poly}(n, d, \log |\mathbb{F}|)$, using n rounds and $O(nd \log |\mathbb{F}|)$ communication.

This resembles UNSAT, since

$$\Phi \in \text{UNSAT} \iff \sum_{x_1, \dots, x_n \in \{0,1\}} \Phi(x_1, \dots, x_n) = 0.$$

There are two differences:

1. the sum may exceed 2 (but we may choose a prime $p > 2^n$);
2. Φ is Boolean, not yet a low-degree polynomial.

From Boolean Formulas to Low-Degree Polynomials

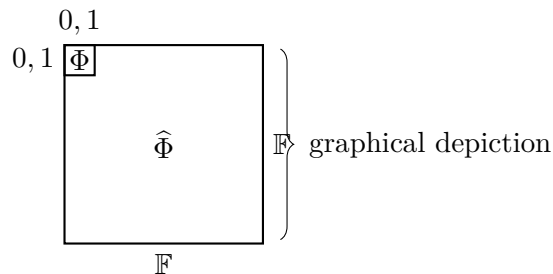
Arithmetization. Given $\Phi : \{0, 1\}^n \rightarrow \{0, 1\}$, construct

$$\hat{\Phi} \in \mathbb{F}^{\leq d}[x_1, \dots, x_n]$$

such that

$$\hat{\Phi}(b_1, \dots, b_n) = \Phi(b_1, \dots, b_n) \quad \text{for every } b_i \in \{0, 1\}.$$

Thus $\hat{\Phi}$ says the same thing as Φ on the Boolean cube and is called a *low-degree extension*.



Apply the following transformations in polynomial time:

$$\begin{aligned} x_v &\longmapsto x_v, \\ \overline{x_v} &\longmapsto 1 - x_v, \\ w \wedge v &\longmapsto wv, \\ w \vee v = \overline{\overline{w} \wedge \overline{v}} &\longmapsto 1 - (1 - w)(1 - v). \end{aligned}$$

The last line is not represented by $w + v$, because the double-positive case would then evaluate to 2.

For the earlier example,

$$\begin{aligned}\Phi &= (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3} \vee x_4), \\ \widehat{\Phi} &= \widehat{c}_1 \widehat{c}_2, \\ \widehat{c}_1 &= 1 - (1 - x_1)x_2(1 - x_3), \\ \widehat{c}_2 &= 1 - (1 - x_2)x_3(1 - x_4).\end{aligned}$$

Moreover,

$$\deg(\widehat{\Phi}) = \sum_{i=1}^m \deg(\widehat{c}_i) = \sum_{i=1}^m 3 \leq 3m.$$

Back to Proving UNSAT

Theorem. Assuming the interactive proof above, $\text{coNP} \subseteq \text{IP}$.

Proof sketch. Given Φ , arithmetize it to obtain

$$\widehat{\Phi} \in \mathbb{F}_p^{\leq 3m}[x_1, \dots, x_n],$$

then run the interactive proof on $\widehat{\Phi}$. The verifier runs in time

$$\text{poly}(n, m, \log |\Phi|) = \text{poly}(|\Phi|).$$

Hence $\text{coNP} \subseteq \text{IP}$. This yields $P^{\#P} \subseteq \text{IP}$; by Toda's theorem, $\text{PH} \subseteq P^{\#P}$, and therefore $\text{PH} \subseteq \text{IP}$. $\square$

September 3, 2026

CS6814 09-03-2026 Notes

Today we wish to show $\text{coNP} \subseteq \text{IP}$ by designing an IP for UNSAT. As a reminder, UNSAT is defined:

$$\text{UNSAT} = \{\Phi \text{ 3-CNF} : \forall x_1, \dots, x_n \in \{0, 1\}, \Phi(x_1, \dots, x_n) = 0\}.$$

Last Time

Given an algorithm that evaluates

$$f : \mathbb{F}^n \rightarrow \mathbb{F}, \quad \mathbb{F} \text{ finite field}, \quad f \in \mathbb{F}^{\leq d}(x_1, \dots, x_n)$$

where superscript $\leq d$ denotes $\deg(f) \leq d$. Previously we used **arithmetisation** to transform

$$\Phi \Rightarrow \hat{\Phi} \in \mathbb{F}^{\leq d}(x_1, \dots, x_n), \quad d \leq 3m$$

and show that for large enough $p > 3m$,

$$\Phi \in \text{UNSAT} \iff \sum_{x_1, \dots, x_n \in \{0, 1\}} \hat{\Phi} = 0.$$

Today

Today, we use the Sumcheck protocol (LFKN'92) to verify that

$$\sum_{b_1, \dots, b_n \in \{0, 1\}} f(b_1, \dots, b_n) = \gamma \in \mathbb{F} \quad \text{using an IP.}$$

which can be extended to an IP for #SAT.

Warmup: $n = 2$ using MA

Suppose

$$\sum_{b_1, b_2 \in \{0, 1\}} f(b_1, b_2) = \gamma.$$

Clearly a brute force approach only needs to evaluate f for $2^2 = 4$ times. However, we imagine an intermediary step for calculating γ , g :

$$g(X) = \sum_{b_2 \in \{0, 1\}} f(X, b_2)$$

where it's clear that

$$g(0) + g(1) = \gamma.$$

We propose the following IP: the prover sends the coefficients $c_0, \dots, c_d$ of a polynomial g :

$$P \xrightarrow{g} V, \quad g(X) = \sum_{i=0}^d c_i X^i, \quad c_i \in \mathbb{F}.$$

The verifier performs the following procedure:

1. Check

$$g(0) + g(1) = \gamma;$$

2. Sample $\alpha \leftarrow \mathbb{F}$ uniformly. Check

$$g(\alpha) = f(\alpha, 0) + f(\alpha, 1);$$

Completeness

Assuming,

$$\sum_{b_1, b_2 \in \{0,1\}} f(b_1, b_2) = \gamma$$

completeness follows rather naturally; (1.) and (2.) are clearly true.

Soundness

$$\sum_{b_1, b_2 \in \{0,1\}} f(b_1, b_2) \neq \gamma.$$

$$\tilde{P} \xrightarrow{\tilde{g}} V, \quad \tilde{g}(X) = \sum_{i=0}^d \tilde{c}_i X^i \quad \deg(\tilde{g}) \leq d.$$

Case 1: $\tilde{g} \equiv g$

By assumption,

$$\tilde{g}(0) + \tilde{g}(1) = g(0) + g(1) = \sum_{b_1, b_2 \in \{0,1\}} f(b_1, b_2) \neq \gamma.$$

thus, we will reject with probability 1.

Case 2: $\tilde{g} \not\equiv g$

$$\tilde{g} \not\equiv g \equiv \sum_{b_2 \in \{0,1\}} f(X, b_2).$$

Now we are interested in

$$\tilde{g}(\alpha) \stackrel{?}{=} g(\alpha).$$

Proposition. For polynomials $p, q \in \mathbb{F}^{\leq d}[X]$, if $p \not\equiv q$, then

$$|\{\alpha \in \mathbb{F} : p(\alpha) = q(\alpha)\}| \leq d.$$

This is equivalent to saying that a degree d polynomial has at most d zeroes. It then follows that

$$\Pr_{\alpha}[\tilde{g}(\alpha) = g(\alpha)] \leq \frac{d}{|\mathbb{F}|}.$$

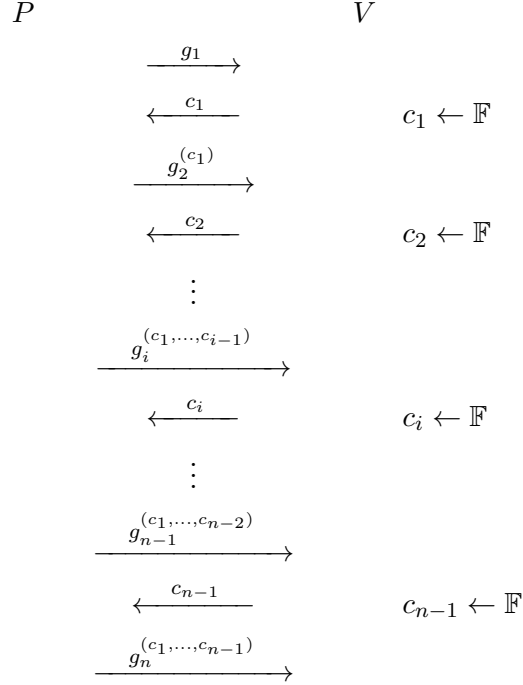
Thus, we reject with probability $\geq 1 - \frac{d}{|\mathbb{F}|} \geq \frac{1}{2}$.

Sumcheck (LFKN'92)

We follow a similar methodology as the $n = 2$ case, taking the i th coordinate to be a free variable.

$$g_i^{(c_1, \dots, c_{i-1})}(X) = \sum_{b_{i+1}, \dots, b_n \in \{0,1\}} f(c_1, \dots, c_{i-1}, X, b_{i+1}, \dots, b_n).$$

Since all g_i are at most degree d , so we can describe each in at most $d + 1$ field elements. We present the following MA[poly] algorithm.



c_i are sampled uniformly from $\mathbb{F}$, and $g_i^{(c_1, \dots, c_{i-1})}$ can be represented or sent over as a list of coefficients. By abuse of notation, sometimes we'll use $g_i^{(c_1, \dots, c_{i-1})} = g_i$.

The verifier then performs the following checks:

1. $g_1(0) + g_1(1) = \gamma$.
2. Sample $c_n \leftarrow \mathbb{F}$ uniformly, then evaluate $g_n(c_n) = f(c_1, \dots, c_n)$.
3. We check $\forall i, g_i(0) + g_i(1) = g_{i-1}(c_{i-1})$.

Completeness

Supposing

$$\sum_{b_1, \dots, b_n \in \{0,1\}} f(b_1, \dots, b_n) = \gamma.$$

Checks (1.) and (2.) obviously pass. For (3.):

$$\begin{aligned}
g_i(0) + g_i(1) &= \sum_{b_i \in \{0,1\}} g_i(b_i) \\
&= \sum_{b_i \in \{0,1\}} \sum_{b_{i+1}, \dots, b_n \in \{0,1\}} f(c_1, \dots, c_{i-1}, b_i, \dots, b_n) \\
&= g_{i-1}^{(c_1, \dots, c_{i-2})}(c_{i-1}).
\end{aligned}$$

Soundness

If the initial sum check passes despite the claimed sum being false— $\sum_{b_1, \dots, b_n \in \{0,1\}} f(b_1, \dots, b_n) \neq \gamma$ but V accepts—then $\tilde{g}_1 \neq g_1$. Conditioned on any history where $\tilde{g}_i \neq g_i$, then we have that

$$\Pr[\tilde{g}_i(c_i) = g_i(c_i)] \leq \frac{d}{|\mathbb{F}|}.$$

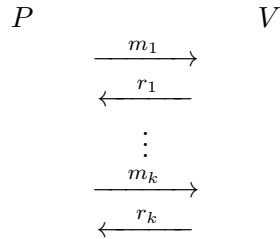
If there isn't such an agreement between the verifier and prover, then passing the next consistency check forces that $\tilde{g}_{i+1} \neq g_{i+1}$, and if this continues through round n , the final evaluation check must reject because of the initial assumption of the sum of f not being equal to γ . It then follows that

$$\Pr[V \text{ accepts false claim}] \leq \frac{nd}{|\mathbb{F}|} \leq \frac{1}{2}$$

for large enough field $\mathbb{F}$.

Limitations of IP

Recall that PSPACE is the set of problems solvable in polynomial space but exponential time. We can show $\text{IP} \subseteq \text{PSPACE}$ with a k -round IP.



So what does this mean exactly? If $L \in \text{IP}$, then

$$x \in L \iff \gamma(x) = \max_P \Pr[\langle P, V(x) \rangle \rightarrow 1] \geq \frac{2}{3}.$$

Completeness:

$$\forall x \in L, \exists P, \Pr[\langle P, V(x) \rangle \rightarrow 1] \geq \frac{2}{3} \implies \gamma(x) \geq \frac{2}{3}.$$

Soundness:

$$\forall x \notin L, \forall P, \Pr[\langle P, V(x) \rangle \rightarrow 1] \leq \frac{1}{3} \implies \gamma(x) \leq \frac{1}{3}.$$

To decide if $x \in L$, compute $\gamma(x)$, output “yes” if $\gamma(x) \geq \frac{2}{3}$, “no” else. **NOTE:** Explicitly iterating over all P ’s not possible in PSPACE. Suppose tr is the transcript of all previous messages.

$$\gamma(x, \text{tr}) = \max_P \Pr[\langle P, V(x) \rangle \rightarrow 1 \mid \text{tr}],$$

then, we can form a tree of possibilities for tr :

$$\begin{cases} \text{tr} = (m_1, r_1, \dots, m_i, r_i) & \text{prover's move,} \\ \text{tr} = (m_1, r_1, \dots, m_i) & \text{verifier's move,} \\ \text{tr} = (m_1, r_1, \dots, m_k, r_k) & \text{protocol done.} \end{cases}$$

And consequently, we can define γ in a kind of recursive way:

$$\gamma(x, \text{tr}) = \begin{cases} \max_m \gamma(x, \text{tr} \| m) & \text{prover move,} \\ \mathbb{E}_r \gamma(x, \text{tr} \| r) & \text{verifier move,} \\ V(x, \text{tr}) & \text{protocol done.} \end{cases}$$

And clearly this is space efficient as we do not need to memoize previous work.

September 8, 2026

CS 6814 Lecture Scribe

Lecturer: Nick Spooner Scriber: Jiucheng Dai

1 Summary

In this lecture, we proved $\text{IP} = \text{PSPACE}$, where

$$\text{PSPACE} = \{\mathcal{L} \text{ that can be decided by a Turing machine in } \text{poly}(n) \text{ space}\}.$$

We will first show $\text{IP} \subseteq \text{PSPACE}$ and then the opposite.

2 $\text{IP} \subseteq \text{PSPACE}$

Consider a language $\mathcal{L} \in \text{IP}$, corresponding to a verifier V . Then $x \in \mathcal{L}$ if and only if

$$\gamma(x) := \max_P \Pr[\langle P, V(x) \rangle \rightarrow 1] \geq 2/3$$

where $\langle P, V(x) \rangle$ is the interaction between the prover P and verifier V , which is defined by a sequence of messages $(m_1, r_1, m_2, r_2, \dots)$. Each m_i is a message the prover sends to the verifier, determined by $(m_1, \dots, r_{i-1})$, and r_i is a random message that the verifier sends to the prover determined by $(m_1, \dots, r_{i-1}, m_i)$ and a random bit r sampled at the start.¹

Then it's enough to compute γ in $\text{poly}(n)$ space.

Also, we can assume this is using public coin by [GS86], meaning that each r_i is a uniformly random string.

For a transcript tr , it can be in 3 forms:

1. when it is the verifier's move: $tr = (m_1, r_1, \dots, m_{i-1}, r_{i-1}, m_i)$
2. when it is the prover's move: $tr = (m_1, r_1, \dots, m_i, r_i)$
3. when the transcript is full: $tr = (m_1, r_1, \dots, m_k, r_k)$

We define further that

$$\gamma(x, tr) = \begin{cases} \max_m \gamma(x, (tr, m)) & \text{at prover's move} \\ \mathbb{E}_r[\gamma(x, (tr, r))] & \text{at verifier's move} \\ V(x, tr) & \text{when transcript is full} \end{cases}$$

Then we define the algorithm of computing $\gamma(x, tr)$ as follows:

1. If it is a prover move: set $\gamma_{\max} = 0$, for each $m \in \{0, 1\}^l$ compute $\gamma(x, (tr, m))$ and update $\gamma_{\max}$ with $\max(\gamma_{\max}, \gamma(x, (tr, m)))$. Return $\gamma_{\max}$ after the recursion.
2. If it is the verifier move: set $\gamma_{\text{avg}} = 0$, for each $r \in \{0, 1\}^l$, compute $\gamma(x, (tr, r))$ and update γ_{avg} with $\gamma_{\text{avg}} + \gamma(x, (tr, r))$. Return γ_{avg} after the recursion.

¹Besides lecture, I also referenced [AB09] for this definition.

3. If tr is the full transcript, return $V(x; tr)$.

Notice that if the transcript is full, the algorithm uses space polynomial in n (notice that $|tr|$ is also polynomial in n and V is a polynomial time verifier). If the transcript is not full, $S(\gamma(x, tr)) = S(\gamma(x, (tr, m))) + \text{poly}(n)$. Since the number of rounds of communication is $\text{poly}(n)$, the total space needed for $S(\gamma(x, tr)) = \text{poly}(n)$.

3 PSPACE $\subseteq$ IP

Consider the true quantified boolean formula

$$\text{TQBF} := \{\text{QBF } \Psi \text{ such that } \Psi \text{ is true}\}$$

where QBF has the form $\forall x_1, \exists x_2, \forall x_3, \exists x_4 \cdots \Phi(x_1, \dots, x_n)$, where Φ is a 3-CNF.

Example 3.1. $\forall x_1, \exists x_2 (x_1 \vee x_2) \in \text{TQBF}$, while $\forall x_1, \exists x_2 (x_1 \wedge x_2) \notin \text{TQBF}$.

Theorem 3.2 ([JG79]). *TQBF is PSPACE complete*

The proof of this can be found in [AB09] Theorem 4.13.

Theorem 3.3 ([Sha92], [Reference for Shen'92 ?](#)).

$$\text{TQBF} \in \text{IP}$$

For a TQBF

$$\forall x_1, \exists x_2, \forall x_3, \exists x_4 \cdots \Phi(x_1, \dots, x_n),$$

we can write it as

$$\bigwedge_{x_1 \in \{0,1\}} \bigvee_{x_2} \bigwedge_{x_3} \bigvee_{x_4} \cdots \Phi(x_1, \dots, x_n)$$

by the algebrization of 3CNF we discussed in last lecture,² there exists a prime p and a polynomial $\hat{\Phi} \in \mathbb{F}_p[x_1, \dots, x_n]$ such that for all $x_1, \dots, x_n \in \{0, 1\}$ $\Phi(x_1, \dots, x_n) = \hat{\Phi}(x_1, \dots, x_n)$, and then we see that the truth value of

$$\bigwedge_{x_1 \in \{0,1\}} \bigvee_{x_2} \bigwedge_{x_3} \bigvee_{x_4} \cdots \Phi(x_1, \dots, x_n)$$

is the same as

$$\prod_{x_1 \in \{0,1\}} \prod_{x_2} \prod_{x_3} \prod_{x_4} \cdots \hat{\Phi}(x_1, \dots, x_n),$$

where

$$\prod_{x_i} f(x_1, \dots, x_i) = f(x_1, \dots, x_{i-1}, 0) f(x_1, \dots, x_{i-1}, 1)$$

²where basically we transform $X \wedge Y$ into $X \cdot Y$ and $X \vee Y$ into $1 - (1 - X)(1 - Y)$

and

$$\prod_{x_i} f(x_1, \dots, x_i) = 1 - (1 - f(x_1, \dots, x_{i-1}, 0))(1 - f(x_1, \dots, x_{i-1}, 1)).$$

Now if we simply do something similar to the sumcheck algorithm we discussed previously, i.e. we define

$$g_1(X) = \prod_{x_2} \prod_{x_3} \prod_{x_4} \dots \hat{\Phi}(X, \dots, x_n) \in \mathbb{F}_p,$$

then the degree of g_n may be exponential in n . Then we made the key observation that for $x_i \in \{0, 1\}$, $(x_i)^e = x_i$ for all positive integer e (for example $x_1^{100} x_2^{50} = x_1 x_2$ for $x_i \in \{0, 1\}$). Then we have the degree reduction operator R , where

$$\begin{aligned} R_{x_i} f(x_1, \dots, x_n) &= f(x_1, \dots, x_{i-1}, 0, x_{i+1}, \dots, x_n) \\ &\quad + x_i (f(x_1, \dots, x_{i-1}, 1, x_{i+1}, \dots, x_n) - f(x_1, \dots, x_{i-1}, 0, x_{i+1}, \dots, x_n)) \end{aligned}$$

This is basically reducing the degree of x_i in each monomial to 1. Then since the R operator does not change the value of the polynomial when x_i takes $\{0, 1\}$, we observe that

$$\prod_{x_1 \in \{0, 1\}} \prod_{x_2} \prod_{x_3} \prod_{x_4} \dots \hat{\Phi}(x_1, \dots, x_n) = \prod_{x_1 \in \{0, 1\}} R_{x_1} \prod_{x_2} R_{x_1} R_{x_2} \prod_{x_3} R_{x_1} R_{x_2} R_{x_3} \dots \hat{\Phi}(x_1, \dots, x_n)$$

Let the operators in the new expression be called $O_1, \dots, O_l$ where $l = \mathcal{O}(n^2)$.

We define the IP as follows. In round 1, the prover should compute $O_1 \dots O_l \hat{\Phi} = \gamma_1$ the verifier checks if it is 1.

Then in round i for $i \geq 1$,

1. If $O_i = \prod_{x_j}$ then the prover will compute $g_i(x) = (O_{i+1} \dots O_l \hat{\Phi})(c_1, \dots, c_{j-1}, x)$ and $\gamma_i = (O_i \dots O_l \hat{\Phi})(c_1, \dots, c_{j-1})$ then verifier checks if $g_i(0)g_i(1) = \gamma_i$ and if $g_{i-1}(c_{j^\dagger}) = \gamma_i$ ($c_{j^\dagger}$ is the c_j sampled in $(i-1)$ th verifier move), if so send $c_j \sim \mathbb{F}_p$ to the prover, else reject.
2. If $O_i = \prod_{x_j}$ then the prover will compute $g_i(x) = (O_{i+1} \dots O_l \hat{\Phi})(c_1, \dots, c_{j-1}, x)$ and $\gamma_i = (O_i \dots O_l \hat{\Phi})(c_1, \dots, c_{j-1})$ then verifier checks if $1 - (1 - g_i(0))(1 - g_i(1)) = \gamma_i$ and if $g_{i-1}(c_{j^\dagger}) = \gamma_i$ ($c_{j^\dagger}$ is the c_j sampled in $(i-1)$ th verifier move), if so send $c_j \sim \mathbb{F}_p$ to the prover, else reject.
3. If $O_i = R_{x_{j'}}$ then the prover will compute $g_i(x) = (O_{i+1} \dots O_l \hat{\Phi})(c_1, \dots, c_{j'-1}, x, c_{j'+1}, \dots, c_j)$ and $\gamma_i = (O_i \dots O_l \hat{\Phi})(c_1, \dots, c_j)$ then verifier checks if $g_i(0) + c_{j'}(g_i(1) - g_i(0)) = \gamma_i$ and if $g_{i-1}(c_{j^\dagger}) = \gamma_i$ ($c_{j^\dagger}$ is the c_j sampled in $(i-1)$ th verifier move), if so send a new $c_{j'} \sim \mathbb{F}_p$ to the prover, else reject.

Completeness is quite direct, since the prover can just compute whatever they should compute. Then we will prove soundness by checking if claim i is false, then claim $i+1$ is false with high probability, where claim i is just the claim that $(O_i \dots O_l \hat{\Phi})(c_1, \dots, c_j) = \gamma_i$.

Suppose in step i $(O_i \dots O_l \hat{\Phi})(c_1, \dots, c_j) \neq \gamma_i$, then denote $\tilde{g}_i$ to be the polynomial the prover sends to the verifier

1. If $\tilde{g}_i = g_i$ then the verifier will reject (for example if $O_i = \prod_{x_j}$ then it will check $g_i(0)g_i(1) = \gamma_i$ and reject)

2. If $\tilde{g}_i \neq g_i$ and the verifier accepts in step $i + 1$, and if claim $i + 1$ is true, which means $(O_{i+1} \cdots O_l \hat{\Phi})(c_1, \dots, c_{j^*}) = \gamma_{i+1}$,³ then $g_i(c_{j^\dagger}) = (O_{i+1} \cdots O_l \hat{\Phi})(c_1, \dots, c_{j^*}) = \gamma_{i+1} = \tilde{g}_i(c_{j^\dagger})$, where $c_{j^\dagger}$ is the c_j sampled in i th verifier move. However this happens with probability less than or equal to $\frac{\deg(g_i)}{|\mathbb{F}_p|}$.

We can also observe that since the verifier only evaluates functions g_i and samples in $\mathbb{F}_p$, it takes time $\text{poly}(\sum_{i=1}^l \deg(g_i), \log |\mathbb{F}_p|)$.

Therefore, we just have to bound the sum $\sum_{i=1}^l \deg(g_i)$. Notice that previously we proved $\deg(g_i) \leq 3m$ where m is the number of clauses in Φ , and notice that when performing the R operator, the degree only decreases, and before performing the $\prod$ or $\coprod$ operator, the degree is always $\leq n$. Then after these operator, the degree is at most $2n$. Therefore $\deg(g_i) \leq \max(3m, 2n)$ for all i . Then since $l = \text{poly}(n)$ we have $\sum_{i=1}^l \deg(g_i) = \text{poly}(n, m)$. If we choose p large enough, we will have that the probability that the verifier accepts a false solution is smaller or equal to $\frac{\sum_i \deg(g_i)}{|\mathbb{F}_p|} = o(1)$ and the verifier has complexity $\text{poly}(\sum_{i=1}^l \deg(g_i), \log |\mathbb{F}_p|) = \text{poly}(m, n)$. Therefore soundness is proven and $\text{TQBF} \in \text{IP}$.

³Here j^* might be j or $j + 1$ depending on O_i

September 10, 2026

Lecture 6

CS 6814: Probabilistic Proofs Fall 2026

Lecturer: Nick Spooner Scribe: Gordon Mei

1 Introduction

In an interactive proof (IP) setting, the verifier's coins are private. In an AM protocol, we use public coins, meaning that the prover knows exactly what the verifier is going to check. In this lecture, we show that with only 2 extra messages, we can go from a private coin protocol to a public coin protocol.

Theorem 1.1. [?] For every $k \in \text{poly}(n)$, $\text{IP}(k) \subseteq \text{AM}(k+2)$.

During this lecture, we prove a relaxed version of Theorem 1.1 for the special case $k = 2$. The main tool is the set lower bound protocol, which is built from pairwise independent hashing. Combining this result with the collapse of constant-round AM, we have the following results:

Theorem 1.2 ([?]). For every constant $k \geq 2$, $\text{AM}[k] = \text{AM}[2]$.

Corollary 1.3. For every constant k , $\text{IP}[k] \subseteq \text{AM}[2]$.

2 Set Lower Bound Protocol

We use the set lower bound protocol to convince a verifier that a set S is large; more precisely, to distinguish the case where S is large from the case where S is small. Membership in S is verifiable in polynomial time with a witness.

For example, consider a language $L \in \text{NP}$, function $K : \mathbb{N} \rightarrow \mathbb{N}$, and let $L_n = L \cap \{0, 1\}^n$. We want a protocol between P and V , such that the following conditions hold:

1. Completeness: if $|L_n| \geq K(n)$, there exists a prover $\mathcal{P}$ such that $\Pr[\langle \mathcal{P}, V \rangle \text{ accepts}] \geq 2/3$.
2. Soundness: if $|L_n| \leq \frac{K(n)}{2}$, for all $\tilde{P}$, $\Pr[\langle \tilde{P}, V \rangle \text{ accepts}] < 1/3$.

2.1 Pairwise Independent Hashing

Definition 2.1. A family of functions $H_{n,k} \subseteq \{0, 1\}^n \rightarrow \{0, 1\}^k$ is pairwise independent if for all $x \neq x' \in \{0, 1\}^n$ and all $y, y' \in \{0, 1\}^k$,

$$\Pr_{h \leftarrow H_{n,k}} [h(x) = y \wedge h(x') = y'] = 2^{-2k}$$

where h is sampled uniformly from $H_{n,k}$.

In other words, on any two distinct inputs, h looks like a truly random function, where the event that both pairs are equal are independent events.

Theorem 2.2. For all $n, k \in \mathbb{N}$, there is a pairwise independent family $H_{n,k}$ whose functions are computable in time $\text{poly}(n, k)$.

Proof. First, we take $k = n$. For all $n \in \mathbb{N}$, let

$$H_{n,n} = \{h_{a,b} : a, b \in \mathbb{F}_{2^n}\}, \quad h_{a,b}(x) = ax + b \in \mathbb{F}_{2^n}$$

Fix $x \neq x'$ and y, y' . Then, $h_{a,b}(x) = y$ and $h_{a,b}(x') = y'$ if and only if

$$a = \frac{y - y'}{x - x'}, \quad b = y - ax$$

Thus, only one of the 2^{2n} pairs (a, b) works, and the probability is 2^{-2n} as required. If $k < n$, we truncate the output to its first k bits. If $k > n$, we pad the input with $k - n$ zeros. $\square$

2.2 Protocol Construction

With these definitions in mind, we are now ready to give our protocol. Fix n , and write $K = K(n)$.

1. V samples $h \leftarrow H_{n,k}$ for k such that $2^{k-2} \leq K \leq 2^{k-1}$ and $y \leftarrow \{0, 1\}^k$, and sends (h, y) .
2. P finds some $x \in L_n$ such that $h(x) = y$ and witness w for $x \in L_n$ (this is simply an NP-witness since $L_n \in \text{NP}$), and sends (x, w) .
3. V accepts if $x \in L_n$ and $h(x) = y$.

Claim 2.3. Define $p = \frac{|L_n|}{2^k}$. Then

$$p \geq \Pr_{\substack{h \leftarrow H_{n,k} \\ y \leftarrow \{0,1\}^k}} [\exists x \in L_n, h(x) = y] \geq p \left(1 - \frac{p}{2}\right)$$

Proof. For the upper bound, start by fixing h . We have that

$$\Pr_y [\exists x \in L_n : h(x) = y] = \Pr_y [L_n \cap h^{-1}(y) \neq \emptyset] = \frac{1}{2^k} \sum_y \mathbb{1}[L_n \cap h^{-1}(y) \neq \emptyset]$$

Since $\mathbb{1}[A \neq \emptyset] \leq |A|$ for any finite set A , and summing $|S \cap h^{-1}(y)|$ over all y counts each element of S exactly once,

$$\frac{1}{2^k} \sum_y \mathbb{1}[L_n \cap h^{-1}(y) \neq \emptyset] \leq \frac{1}{2^k} \sum_y |L_n \cap h^{-1}(y)| = \frac{|L_n|}{2^k} = p$$

We now prove the lower bound. Fix y , and denote the event that $h(x) = y$ as E_x .

$$\Pr \left[\bigcup_{x \in L_n} E_x \right] \geq \sum_{x \in L_n} \Pr[E_x] - \frac{1}{2} \sum_{x \neq x' \in L_n} \Pr[E_x \cap E_{x'}]$$

We use inclusion-exclusion for the first inequality, where the $1/2$ accounts for each unordered pair being counted twice in the second sum. By pairwise independence, $\Pr[E_x] = 2^{-k}$ and $\Pr[E_x \cap E_{x'}] = 2^{-2k}$, so

$$\Pr \left[\bigcup_{x \in L_n} E_x \right] \geq |L_n| \cdot 2^{-k} - \frac{1}{2} |L_n|^2 \cdot 2^{-2k} = p - \frac{p^2}{2}.$$

$\square$

Let $p^* = \frac{K}{2^k}$. By our choice of k , $\frac{1}{4} \leq p^* \leq \frac{1}{2}$.

Claim 2.4. *The protocol has completeness $\alpha = \frac{3}{4}p^*$ and soundness $\beta = \frac{1}{2}p^*$.*

Proof. We prove completeness. Suppose $|L_n| \geq K(n)$. We assume WLOG $|L_n| = K(n)$. Then

$$\Pr [\exists x \in L_n : h(x) = y] \geq p \left(1 - \frac{p}{2}\right) = p^* \left(1 - \frac{p^*}{2}\right) \geq \frac{3}{4}p^*$$

We now turn to soundness. Suppose $|L_n| \leq \frac{K(n)}{2}$, then by the previous claim, the verifier accepts with probability $\leq p \leq \frac{p^*}{2}$. $\square$

For completeness α and soundness β , we have a gap $\alpha - \beta \geq \frac{3}{4}p^* - \frac{1}{2}p^* \geq \frac{1}{16}$ with $p^* \in [\frac{1}{4}, \frac{1}{2}]$. Thus, we can amplify the gap to $(\frac{2}{3}, \frac{1}{3})$ by repeating the protocol $O(1)$ times in parallel, accepting if at least $\frac{\alpha+\beta}{2}$ fraction of the copies accept. With Chernoff bounds, we will then get the necessary completeness and soundness errors.

3 IP[2] $\subseteq$ AM[5]

We show a weaker version of the private to public coin protocol in the $k = 2$ case.

3.1 Setup

Let $L \in \text{IP}[2]$. For input $x \in \{0, 1\}^n$, the private-coin protocol for L is:

1. V samples $r \leftarrow \{0, 1\}^R$, where $R = R(n) \in \text{poly}(n)$, and sends $q = Q(x, r)$, for some polynomial-time query function Q .
2. P computes an answer a and sends it.
3. V accepts if $V(x, r, a) = 1$.

Define the sets

$$R_q = \{r : Q(x, r) = q\}, \quad S_{qa} = \{r \in R_q : V(x, r, a) = 1\}$$

R_q is the set of random tapes that lead to question q , and S_{qa} is the set of those tapes on which answer a is accepted. Given q , the verifier's tape is uniform on R_q , so the best prover answers q with an a maximizing $|S_{qa}|/|R_q|$. The maximum acceptance probability is

$$\gamma(x) = \max_{\tilde{P}} \Pr [\langle \tilde{P}, V \rangle \text{ accepts}] = \mathbb{E}_{r, q=Q(x, r)} \left[\max_a \frac{|S_{qa}|}{|R_q|} \right]$$

We want to show that for most r , there exists an a such that $|S_{Q(x, r), a}|$ is large. For all $N \in \mathbb{N}$, we define:

$$R_N = \{r : \exists a |S_{Q(x, r), a}| > N\}$$

that is, for the question $Q(x, r)$ generated by $r \in R_N$, there exists an answer a that is accepted on more than N random tapes.

For our public coin protocol, we want to show that R_N is large. Let $0 < \varepsilon < 1$, and let γ be a completeness threshold, so $\gamma(x) \geq \gamma$ for $x \in L$.

1. P finds and sends some N such that $|R_N| \geq \frac{\gamma\varepsilon \cdot 2^R}{N}$ (such an N exists by Markov).
2. V sends $h_1 \leftarrow H_{n,k}$, $y_1 \leftarrow \{0,1\}^k$.
3. P sends $r \in R_N$ and a such that $h_1(r) = y_1$ and a is a witness for R_N .
4. V checks $h_1(r) = y_1$ and $r \in R_N$, i.e., $|S_{Q(x,r),a}| > N$. It sends $h_2 \leftarrow H_{R,k}$ and $y_2 \leftarrow \{0,1\}^k$.
5. P sends $r' \in S_{Q(x,r),a}$ with $h_2(r') = y_2$.

Finally, V accepts if $h_2(r') = y_2$ and $V(x, r, a; r') = 1$.

3.2 Soundness

The completeness argument is straightforward, so we focus on the soundness of our protocol. Suppose the claim P makes in step 1 holds, i.e. $|R_N| \geq \frac{\gamma\varepsilon \cdot 2^R}{N}$. We claim that this implies that the original private-coin protocol has acceptance probability at least $\gamma\varepsilon$.

Proof. Expanding the definition of $\gamma(x)$ and writing $q = Q(x, r)$,

$$\begin{aligned}
\gamma(x) &= \mathbb{E}_r \left[\frac{\max_a |S_{qa}|}{|R_q|} \right] = \frac{1}{2^R} \sum_r \frac{1}{|R_q|} \cdot \max_a |S_{qa}| \\
&\geq \Pr_r[r \in R_N] \cdot \mathbb{E}_r \left[\frac{\max_a |S_{qa}|}{|R_q|} \mid r \in R_N \right] \\
&\geq \Pr_r[r \in R_N] \cdot N \\
&= \frac{\gamma\varepsilon}{N} \cdot N = \gamma\varepsilon
\end{aligned}$$

The first inequality drops the terms with $r \notin R_N$, all of which are non-negative. The second holds because every $r \in R_N$ has some a with $|S_{Q(x,r),a}| > N$. The last line uses $\Pr_r[r \in R_N] = \frac{|R_N|}{2^R} \geq \frac{\gamma\varepsilon}{N}$. $\square$

Taking the contrapositive, if $\gamma(x) < \gamma\varepsilon$, then no N satisfies the bound claimed in step 1, so $|R_N|$ is smaller than the claimed $\frac{\gamma\varepsilon \cdot 2^R}{N}$ and the set lower bound protocol rejects. Steps 2–3 of the protocol apply it to R_N and steps 4–5 apply it to $S_{Q(x,r),a}$, with the inner invocation serving as the check that $r \in R_N$. Amplifying each set lower bound invocation $O(1)$ times in parallel gives the required soundness error, and since parallel repetition does not add rounds, the protocol uses 5 messages, giving $\text{IP}[2] \subseteq \text{AM}[5]$.

Remark 3.1 (General k). *For general k , $\gamma(x)$ is an alternating sequence of expectations (over the verifier's messages) and maximums (over the prover's messages). We continue to apply the same idea: the prover makes claims about the sizes of the relevant sets of tapes, and the verifier checks them with set lower bounds, applied sequentially round by round.*

September 15, 2026

6.814: Probabilistic Proofs
Lecture Notes

Scribe: Davey Seeman

Today, we (1) show that Graph Non-Isomorphism is in AM, and (2) begin studying *doubly-efficient interactive proofs*, culminating in the start of the GKR protocol.

1 Graph Non-Isomorphism is in AM

Definition 1 (Graph Non-Isomorphism).

$$\text{GNI} = \{(G_0, G_1) : G_0 \not\cong G_1\}.$$

Theorem 2. $\text{GNI} \in \text{AM}$.

Proof. Let (G_0, G_1) be a pair of n -vertex graphs, which we may assume both have vertex set $[n]$. Our goal is to use an AM protocol to determine whether G_0 and G_1 are isomorphic. For $b \in \{0, 1\}$, let

$$\text{aut}(G_b) = \{\pi : \pi(G_b) = G_b\} \quad \text{and} \quad S_b = \{H : H \cong G_b \text{ and } H \text{ has vertex set } [n]\},$$

where π ranges over permutations of $[n]$.

A simpler case. First suppose that $\text{aut}(G_b) = \{\text{id}\}$ for each b . Then $|S_b| = n!$ since all isomorphic graphs are just permutations of the vertex set. Define

$$S = S_0 \cup S_1.$$

We consider the two cases:

- *Yes case* ($G_0 \not\cong G_1$, i.e., the graphs are non-isomorphic): then $S_0 \cap S_1 = \emptyset$, so $|S| = 2 \cdot n!$.
- *No case* ($G_0 \cong G_1$): then $S_0 = S_1$, so $|S| = n!$.

The verifier and prover then run the Goldwasser–Sipser set lower bound protocol on S to determine if $G_0 \not\cong G_1$. This is because, as we learned last class, the Goldwasser–Sipser protocol guarantees that the verifier accepts with high probability if the set S has size at least m , and rejects with high probability if S has size at most $m/2$. (In this case, $m = 2 \cdot n!$.) The protocol requires that membership in S can be efficiently verified: here, the prover proves $H \in S$ by sending a bit b and a permutation σ , and the verifier checks that $\sigma(G_b) = H$. Since Goldwasser–Sipser is an AM protocol, this establishes the proof for the simpler case.

The general case. Dropping the assumption on the automorphism groups, we have in general

$$|S_b| = \frac{n!}{|\text{aut}(G_b)|}.$$

This follows from the orbit–stabilizer theorem: the permutation group S_n acts on graphs with vertex set $[n]$, the orbit of G_b is S_b , and the stabilizer of G_b is $\text{aut}(G_b)$, so $|S_b| \cdot |\text{aut}(G_b)| = |S_n| = n!$. The next step is to construct a set S'_b with $|S'_b| = n!$, for which we can still efficiently check membership and which has the same properties listed above in the yes and no cases. Define

$$S'_b = \{(H, \pi) : H \in S_b, \pi \in \text{aut}(H)\}.$$

Since isomorphic graphs have automorphism groups of the same size, $|S'_b| = |S_b| \cdot |\text{aut}(G_b)| = n!$. Checking membership in S'_b is efficient, since the prover can send an isomorphism showing $H \in S_b$ (as before), and we can apply π to H to see whether it is an automorphism. Now let's check the properties from before. Let $S' = S'_0 \cup S'_1$.

- *Yes case* ($G_0 \not\cong G_1$, i.e., the graphs are non-isomorphic): then $S_0 \cap S_1 = \emptyset$, so $S'_0 \cap S'_1 = \emptyset$, so $|S'| = 2 \cdot n!$.
- *No case* ($G_0 \cong G_1$): then $S'_0 = S'_1$ since $S_0 = S_1$, so $|S'| = n!$.

Thus we can now run the Goldwasser–Sipser protocol on S' to find out whether the graphs are non-isomorphic, so $\text{GNI} \in \text{AM}$. $\square$

Theorem 3. *If GI is NP-complete, then the polynomial hierarchy collapses to its second level ($\text{PH} = \Sigma_2^P$). This is evidence that GI is not NP-complete.*

Proof sketch.

$$\text{GI is NP-complete} \implies \text{GNI is coNP-complete} \implies \text{coNP} \subseteq \text{AM} \implies \text{PH} = \Sigma_2^P.$$

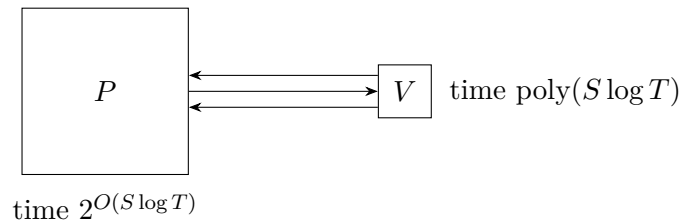
The first implication holds since GNI is the complement of GI. The second implication holds because every coNP language reduces to GNI, which is in AM by Theorem 2, and AM is closed under such reductions. The third is a known result: $\text{coNP} \subseteq \text{AM}$ implies that the polynomial hierarchy collapses. The collapse of PH is generally believed to be false, which is why this is viewed as evidence that GI is not NP-complete. $\square$

2 Doubly-Efficient Interactive Proofs

Recall from last class that deciding whether a computation accepts can be reduced to deciding whether a certain TQBF instance (true quantified Boolean formula) is true. More precisely, for a machine M running in time T and space S , there is an efficient transformation mapping each input x to a quantified Boolean formula φ_x with $O(S \log T)$ variables such that

$$M \text{ accepts } x \iff \varphi_x \text{ is true.}$$

Running the $\text{IP} = \text{PSPACE}$ protocol on such a TQBF instance looks like this:



- For $L \in \text{PSPACE}$: $T = 2^{\text{poly}(n)}$ and $S = \text{poly}(n)$. The prover's complexity is exponential in n , and the verifier's complexity is polynomial in n . Indeed, $S \log T = \text{poly}(n) \cdot \text{poly}(n) = \text{poly}(n)$, so the prover runs in time $2^{O(S \log T)} = 2^{\text{poly}(n)}$ and the verifier runs in time $\text{poly}(S \log T) = \text{poly}(n)$.
- For $L \in \text{L (LOGSPACE)}$: $T = \text{poly}(n)$ and $S = O(\log n)$, so $S \log T = O(\log^2 n)$. The verifier's running time is $n \cdot \text{poly}(\log n)$, which is very good. But the prover's running time is bad: $2^{O(\log^2 n)} = n^{O(\log n)}$.

We would like the running time of the prover to also be polynomial. This leads to the idea of *doubly-efficient interactive proofs*.

Definition 4 (Doubly-efficient interactive proofs). delP is the class of languages with an interactive proof in which the prover runs in polynomial time and the verifier runs in near-linear time $\tilde{O}(n)$.

Open Question 5. Which languages have doubly-efficient interactive proofs?

There are two easy containments:

$$\text{BPTIME}(n) \subseteq \text{delP} \subseteq \text{BPP},$$

where $\text{BPP} = \bigcup_{c \geq 1} \text{BPTIME}(n^c)$. Recall that $\text{BPTIME}(t(n))$ is the class of languages decided by a probabilistic algorithm running in time $O(t(n))$ that errs with probability at most $1/3$ on every input. If $L \in \text{BPTIME}(n)$, the verifier simply runs this $O(n)$ -time algorithm and ignores the prover, so it runs in time $O(n) \subseteq \tilde{O}(n)$.

Recall that a language is in BPP if some polynomial-time randomized algorithm gives the correct answer on every input with probability at least $2/3$. For the second containment, suppose $L \in \text{delP}$, and let P and V be the prover and verifier of the protocol. A BPP algorithm for L would work as follows: on input x , it flips the verifier's coins, computes each message of P and of V in turn, and finally outputs whatever V decides. This takes polynomial time, because P runs in polynomial time and V runs in time $\tilde{O}(n)$. The algorithm is correct with probability at least $2/3$ on every input since if $x \in L$, then by completeness, V accepts when interacting with P with probability at least $2/3$, so the algorithm outputs “yes” with probability at least $2/3$. If $x \notin L$, then by soundness, no prover can make V accept with probability more than $1/3$. In particular, this holds for P , so the algorithm outputs “yes” with probability at most $1/3$, i.e., it outputs the correct answer “no” with probability at least $2/3$.

We will now prove the following.

Theorem 6. $\text{L} \subseteq \text{delP}$.

The proof is via the GKR protocol.

Aside. We will show $\text{L} \subseteq \text{delP}$, but the general theorem is that any language L computable by a family of $O(\log n)$ -space uniform circuits of size T and depth d has an interactive proof with prover time $\text{poly}(T)$ and verifier time $n \cdot \text{poly}(d, \log T)$.

Here, an *S -space uniform circuit family* is a family $\{C_n\}_{n \in \mathbb{N}}$, with $C_n : \{0, 1\}^n \rightarrow \{0, 1\}$, such that there exists a Turing machine M using space $S(n)$ with $M(1^n)$ outputting C_n .

3 The GKR Protocol

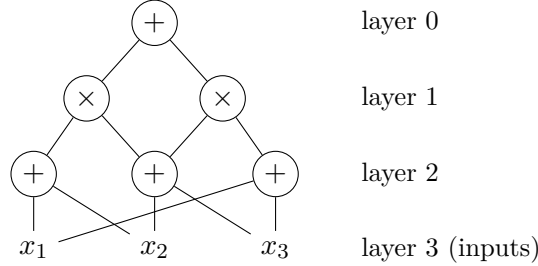
3.1 Setup: circuits

Take $\ell \in \mathsf{L} \subseteq \mathsf{NC}^2$, where

$$\mathsf{NC}^2 = \{\ell : \ell \text{ is computed by uniform poly-size Boolean circuits} \\ \text{of fan-in 2 and depth } O(\log^2 n)\}.$$

The containment $\mathsf{L} \subseteq \mathsf{NC}^2$ is a classical result. In fact, $\mathsf{L} \subseteq \mathsf{NL} \subseteq \mathsf{NC}^2$.

Recall that *Boolean circuits* are directed acyclic graphs with variables on the leaves and operations $\wedge, \vee, \neg$ on the internal nodes, which together define a Boolean function. Fan-in 2 means that each node takes at most 2 inputs. An *arithmetic circuit* is obtained by replacing $\vee$ and $\wedge$ in Boolean circuits with $+$ and $\times$. For example:



This circuit computes $(x_1 + x_2)(x_2 + x_3) + (x_2 + x_3)(x_1 + x_3)$. Every gate takes its two inputs from the layer directly below it.

Since $\ell \in \mathsf{NC}^2$, ℓ can be computed by an arithmetic circuit of depth $d = O(\log^2 n)$.

We will not worry about the details of the conversion between boolean and arithmetic or about the other properties we assume about arithmetic circuits; we state them here explicitly. Over a finite field $\mathbb{F}$, each Boolean gate is replaced by arithmetic operations that agree with it on inputs in $\{0, 1\}$:

$$x \wedge y \mapsto x \cdot y, \quad \neg x \mapsto 1 - x, \quad x \vee y \mapsto x + y - x \cdot y.$$

We also assume, without proof, that the resulting arithmetic circuit

- is *layered*: each gate in layer i takes both of its inputs from layer $i + 1$, where layer 0 is the output gate and layer d is the input layer (as in the example above);
- has fan-in exactly 2, and each layer consists of only one type of gate: addition gates on even layers and multiplication gates on odd layers (as in the example above). This is without loss of generality if the constants 0 and 1 are provided as extra inputs. This is because we can split each layer into an addition layer followed by a multiplication layer. Each gate is computed in the layer of its type, and is passed through the other layer as $u + 0$ or $u \times 1$. This at most roughly triples the number of gates per layer and doubles the depth;
- still has polynomial size and depth $d = O(\log^2 n)$, and is still $O(\log n)$ -space uniform.

Let $\{C_n\}_{n \in \mathbb{N}}$ denote the resulting family of arithmetic circuits, so that for every input $x = (x_1, \dots, x_n) \in \{0, 1\}^n$,

$$C_n(x) = 1 \iff x \in \ell.$$

Our goal is to show that the output of the circuit C_n on input x is 1, which shows that $x \in \ell$. This is exactly what we need: an interactive proof for ℓ is a protocol in which the prover convinces the verifier that a given input x is in ℓ . So if we give a protocol in which an efficient prover convinces a near-linear-time verifier that $C_n(x) = 1$, then $\ell \in \text{delP}$, and since $\ell \in \mathbf{L}$ was arbitrary, this proves Theorem 6.

3.2 Wiring functions and their extensions

We will now state some definitions about our circuit. Define $K = 2^k$ to be an upper bound on the number of gates per layer, where $k = O(\log n)$. Number the gates in each layer $\{0, \dots, K-1\}$. For each layer $i \in \{0, \dots, d-1\}$, define the *wiring function*

$$G_i : [K]^3 \rightarrow \{0, 1\},$$

$$G_i(a, b, c) = 1 \iff \text{gate } a\text{'s first and second inputs are gates } b \text{ and } c, \text{ respectively}$$

in the next layer, for $a, b, c \in [K]$. Since $K = 2^k$, we may identify $[K] \simeq \{0, 1\}^k$, and so we view G_i as a function

$$G_i : \{0, 1\}^{3k} \rightarrow \{0, 1\}.$$

Let $\mathbb{F}$ be a finite field, and let $\widehat{G}_i$ be a degree- D extension of G_i over $\mathbb{F}$; that is,

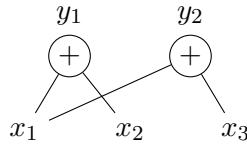
$$\widehat{G}_i : \mathbb{F}^{3k} \rightarrow \mathbb{F}, \quad \deg(\widehat{G}_i) \leq D, \quad \text{and} \quad \widehat{G}_i(a) = G_i(a) \quad \text{for all } a \in \{0, 1\}^{3k}.$$

We make this definition because the protocol will ask the verifier to evaluate the wiring function at random points of $\mathbb{F}^{3k}$, not just at points of $\{0, 1\}^{3k}$. Here D is small (roughly on the order of k , while the field size should be large compared to D). We assume without proof that such an extension exists. For example, the multilinear extension of G_i (see below) has degree at most 1 in each of its $3k$ variables, so its total degree is at most $3k = O(k)$.

For now, we also assume that $\widehat{G}_i$ is efficiently computable, i.e., in time $\text{poly}(\log n)$.

3.3 Gate values and their multilinear extensions

Fix a particular input $(x_1, x_2, \dots, x_n)$ to the circuit. For example, in the circuit



we have $y_1 = x_1 + x_2$ and $y_2 = x_1 + x_3$. We can therefore write the values of all gates, layer by layer, in a table:

	output (layer 0)			
⋮				
y_1 y_2 $\cdots$				
x_1 x_2 $\cdots$ x_n				
input (layer d)				

Call the i -th row of this table V_i , so that

$$V_i : [K] \rightarrow \mathbb{F}, \quad \text{or equivalently} \quad V_i : \{0, 1\}^k \rightarrow \mathbb{F}.$$

(Unused gate labels are assigned the value 0.)

Definition 7. Define $\widehat{V}_i : \mathbb{F}^k \rightarrow \mathbb{F}$ to be the unique multilinear extension of V_i ; that is,

- (1) $\widehat{V}_i(a) = V_i(a)$ for all $a \in \{0, 1\}^k$, and
- (2) the degree of $\widehat{V}_i$ in each variable is at most 1.

Remark. Unlike the extension $\widehat{G}_i$ above, this extension is unique.

It is a standard fact that every function $f : \{0, 1\}^k \rightarrow \mathbb{F}$ has a unique multilinear extension. Existence is shown by the explicit formula

$$\widehat{f}(z) = \sum_{a \in \{0, 1\}^k} f(a) \prod_{j=1}^k (a_j z_j + (1 - a_j)(1 - z_j)).$$

Uniqueness holds because the only multilinear polynomial that vanishes on all of $\{0, 1\}^k$ is the zero polynomial.

3.4 The idea of the protocol

For each layer, we start with a claim of the form

$$\widehat{V}_i(\alpha_i) = \beta_i,$$

and then use interaction to reduce it to a claim of the form

$$\widehat{V}_{i+1}(\alpha_{i+1}) = \beta_{i+1}.$$

We can check the claims about the first and last layers ourselves.

For the first layer (layer 0), the claim is the statement we want to prove: if $\alpha_0 \in \{0, 1\}^k$ is the label of the output gate, then the claim is $\widehat{V}_0(\alpha_0) = V_0(\alpha_0) = 1$, and the verifier sets $\beta_0 = 1$ itself. For the last layer (layer d), V_d is just the input x (padded with zeros), so the verifier can compute $\widehat{V}_d(\alpha_d)$ directly from x using the formula for the multilinear extension. This takes about $O(n \cdot k)$ field operations, which is $\tilde{O}(n)$.

Observation 8. For $a \in \{0, 1\}^k$, if layer i consists of addition gates (i even), we can write

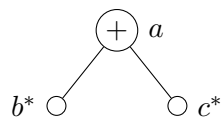
$$V_i(a) = \sum_{b, c \in \{0, 1\}^k} G_i(a, b, c) \cdot (V_{i+1}(b) + V_{i+1}(c)),$$

and if layer i consists of multiplication gates (i odd),

$$V_i(a) = \sum_{b, c \in \{0, 1\}^k} G_i(a, b, c) \cdot V_{i+1}(b) \cdot V_{i+1}(c).$$

Proof. The term $G_i(a, b, c)$ is 1 only when b and c are the wires connected to a . Since gate a has exactly one ordered pair of inputs (b^*, c^*) , every term of the sum is 0 except the one with $(b, c) = (b^*, c^*)$. So, for i even, since layer i consists of addition gates,

$$V_i(a) = V_{i+1}(b^*) + V_{i+1}(c^*).$$



Similarly, for i odd, layer i consists of multiplication gates, and the sum collapses to $V_i(a) = V_{i+1}(b^*) \cdot V_{i+1}(c^*)$. $\square$

This is as far as we got in this lecture. Next time, we will complete the proof of Theorem 6 by showing how the prover and verifier can use interaction to reduce a claim about $\widehat{V}_i$ to a claim about $\widehat{V}_{i+1}$.

AI Disclosure

I sent an unfinished draft to Claude (Anthropic) and asked it to complete the following tasks:

- (1) Drawing TikZ pictures based on the drawings from my notes.
- (2) Explaining to me why the containment $\mathbf{dIP} \subseteq \mathbf{BPP}$ holds.
- (3) Trying to find any errors in my draft. It found that the definition $S'_b = \{(H, \pi) : H \in S_b, \pi \in \text{aut}(G_b)\}$ needed to be replaced with $S'_b = \{(H, \pi) : H \in S_b, \pi \in \text{aut}(H)\}$. It also found that I needed to reduce the arithmetic circuits to those in which each layer consists of only one type of gate so that observation 8 holds and that I needed to assume the arithmetic circuits were layered.
- (4) Explaining why we care about showing the output of the circuit C_n on input x is 1.
- (5) Explaining how the verifier can check the claims of the first and last layer in the arithmetic circuit.
- (6) Explaining why every function $f : \{0, 1\}^k \rightarrow \mathbb{F}$ has a unique multilinear extension.
- (7) Reformatting the LaTeX to look nicer and fixing the grammar in some places.

September 17, 2026

9/17 notes

Will Bradley

Recap

Theorem [GKR'08]: L a.k.a. LOGSPACE has doubly efficient interactive proofs (prover in $\text{poly}(n)$, verifier in $n \text{polylog}(n)$). We transform any $L \in \text{L}$ into an arithmetic circuit with depth $d \in O(\log^2 n)$. How do we do this?

Let $K = 2^k$ be the number of gates per layer (upper bound), and wiring function $G_i : [K]^3 \rightarrow \{0, 1\}$, where $G_i(a, b, c) = 1$ iff gate $\#a$ in layer i is connected to (has inputs) $\#b$ and $\#c$ in layer $i + 1$. We can view G_i as $\{0, 1\}^{3k} \rightarrow \{0, 1\}$, and then a degree- D extension over a finite field $\mathbb{F}$; that is, some $\widehat{G}_i : \mathbb{F}^{3k} \rightarrow \mathbb{F}$ so that $\deg(\widehat{G}_i) \leq D$, and G_i and $\widehat{G}_i$ agree on $\{0, 1\}^{3k}$.

We can encode the output of each gate in layer i as $V_i : [K] \rightarrow \mathbb{F}$, where V_0 is the (single) output and V_d consists of the inputs. (If there are fewer than K gates in a layer, write down some arbitrary value.) We may view V_i as $\{0, 1\}^k \rightarrow \mathbb{F}$, and we can get the unique multilinear extension of V_i , which we call $\widehat{V}_i : \mathbb{F}^k \rightarrow \mathbb{F}$, so that $\widehat{V}_i$ agrees with V_i on $\{0, 1\}^k$ and the degree of any variable in $\widehat{V}_i$ is at most 1.

New stuff

Some string x is in the language L if and only if the arithmetic circuit outputs 1 when given x as input. That is, $V_0(1) = 1$; equivalently, $\widehat{V}_0(00 \dots 0) = 1$. High-level idea: for $\alpha_i, \alpha_{i+1} \in \mathbb{F}^k$, $\beta_i, \beta_{i+1} \in \mathbb{F}$, reduce the claim $\widehat{V}_i(\alpha_i) = \beta_i$ to $\widehat{V}_{i+1}(\alpha_{i+1}) = \beta_{i+1}$. By repeating this process, we can get down to a claim of $\widehat{V}_d(\alpha_d) = \beta_d$, which can be checked in time $n \text{polylog}(n)$. For the even i , so that all the nodes in layer i are $+$, we have

$$V_i(a) = \sum_{\{b, c\} \subseteq [K]} G_i(a, b, c)(V_{i+1}(b) + V_{i+1}(c)),$$

and for the odd i ($\times$ layer), we have

$$V_i(a) = \sum_{\{b, c\} \subseteq [K]} G_i(a, b, c)(V_{i+1}(b)V_{i+1}(c)).$$

Notice these are sums over distinct $b, c \in [K]$. We can try replacing all the V_i, V_{i+1} above with their extensions $\widehat{V}_i, \widehat{V}_{i+1}$, so that

$$\widehat{V}_i(\vec{a}) = \sum_{\{\vec{b}, \vec{c}\} \subseteq \{0, 1\}^k} \widehat{G}_i(\vec{a}, \vec{b}, \vec{c}) \left(\widehat{V}_{i+1}(\vec{b})(+ \text{ or } \times) \widehat{V}_{i+1}(\vec{c}) \right) \quad (*)$$

for $\vec{a} \in \{0, 1\}^k \dots$ but this doesn't work for the entire domain in general, i.e. $\vec{a} \in \mathbb{F}^k$. (If $\widehat{G}_i$ were multilinear like $\widehat{V}_i$, this would hold. A later paper [Cormode, Mitzenmacher, Thaler '12] showed that we can get $\widehat{G}_i$ multilinear, but we will go a different route.) We will make the polynomial inside the sum multilinear. To do this, define the multilinear, bivariate polynomial

$$\text{EQ}(x, y) := 1 - x - y + 2xy;$$

note that for $x, y \in \{0, 1\}$, $\text{EQ}(x, y)$ is 1 if $x = y$ and 0 otherwise. We can generalize this to $2k$ variables:

$$\text{EQ}(\vec{x}, \vec{y}) := \prod_{i=1}^k \text{EQ}(x_i, y_i),$$

so that $\text{EQ}(\vec{x}, \vec{y})$ is a multilinear polynomial that returns 1 if $\vec{x} = \vec{y}$ and 0 otherwise. Therefore, we write

$$\hat{V}_i(\vec{\alpha}) = \sum_{\vec{a}, \vec{b}, \vec{c} \in \{0, 1\}^k} \text{EQ}(\vec{\alpha}, \vec{a}) \widehat{G}_i(\vec{a}, \vec{b}, \vec{c}) \left(\hat{V}_{i+1}(\vec{b})(+ \text{ or } \times) \hat{V}_{i+1}(\vec{c}) \right)$$

for all $\vec{\alpha} \in \mathbb{F}^k$. To see that this is multilinear (and therefore the equation we just wrote is true on the entire field, because multilinear extension is unique), see that it reduces to (*) above for $\vec{\alpha} \in \{0, 1\}^k$, and any multilinear polynomials that agree on $\{0, 1\}^k$ are the same over the entire field.

Now we are ready to describe the protocol. Given a claim that $\hat{V}_i(\alpha_i) = \beta_i$, we saw that this is the same as

$$\beta_i = \sum_{\vec{a}, \vec{b}, \vec{c} \in \{0, 1\}^k} \text{EQ}(\alpha_i, \vec{a}) \widehat{G}_i(\vec{a}, \vec{b}, \vec{c}) \left(\hat{V}_{i+1}(\vec{b})(+ \text{ or } \times) \hat{V}_{i+1}(\vec{c}) \right).$$

The **sumcheck protocol** applies to this form, $\sum_{\vec{x} \in \{0, 1\}^k} f(\vec{x}) = \beta$. (We needed $\hat{V}_i$ to have the recursive relationship we proved above on the whole field so that sumcheck doesn't return a false negative.) At the end of the protocol, we need to evaluate the claim that $f(\gamma_1, \gamma_2, \gamma_3) = \eta$, where $\gamma_1, \gamma_2, \gamma_3 \in \mathbb{F}^k$, $\eta \in \mathbb{F}$, and

$$f(\gamma_1, \gamma_2, \gamma_3) = \text{EQ}(\alpha_i, \gamma_1) \widehat{G}_i(\gamma_1, \gamma_2, \gamma_3) \left(\hat{V}_{i+1}(\gamma_2)(+ \text{ or } \times) \hat{V}_{i+1}(\gamma_3) \right).$$

How long does this take to evaluate? The EQ function takes $\text{poly}(k, \log|\mathbb{F}|) = \text{polylog}(n)$. We will show at some point that we can evaluate $\widehat{G}_i$ in time $\text{polylog}(n)$. To evaluate the two $\hat{V}_{i+1}$ parts, we ask the prover to send

$$\eta_1 := \hat{V}_{i+1}(\gamma_2) \quad \text{and} \quad \eta_2 := \hat{V}_{i+1}(\gamma_3).$$

The prover sends η_1^* and η_2^* , which may or may not be η_1 and η_2 , respectively. After all this, we can check whether

$$\text{EQ}(\alpha_i, \gamma_1) \widehat{G}_i(\gamma_1, \gamma_2, \gamma_3) (\eta_1^*(+ \text{ or } \times) \eta_2^*) = \eta$$

in polynomial time.

The prover might give us incorrect η_1^* or η_2^* , but then we could go check whether $\eta_1^* = \eta_1$ and $\eta_2^* = \eta_2$ (recursively). Unfortunately, the number of calls increases by a factor of 2 at each step, so the total number of prover calls is order of $2^d = 2^{\log^2 n} = n^{\log n}$. To solve this, we use the **query reduction** protocol (Kalai–Raz '08, but elsewhere in the literature as well).

Query reduction protocol

Consider some $f \in \mathbb{F}^{\leq d}[X_1, \dots, X_n]$. The prover and verifier have oracle access to evaluating f . Consider $(\alpha_1, \beta_1), \dots, (\alpha_t, \beta_t)$, where each $\alpha_i \in \mathbb{F}^n$ and each $\beta_i \in \mathbb{F}$. Say we want to check that for all $i \in [t]$, $f(\alpha_i) = \beta_i$. (For the previous part, $t = 2$.) We can do this by evaluating f t many times (at each of the α_i), but we can do better (in terms of queries to f). Define $g : \mathbb{F} \rightarrow \mathbb{F}^n$ as

$$g(\alpha) := (g_1(\alpha), \dots, g_n(\alpha)),$$

where we choose $g_i(\alpha)$ to be polynomials of minimum degree such that $g(i) = \alpha_i$ for all $i \in [t]$. It turns out that that degree is $\leq t - 1$. Let $h : \mathbb{F} \rightarrow \mathbb{F} := f \circ g$. The degree of h is at most

$(\deg f)(\deg g) \leq dnt$. Additionally, for all $i \in [t]$, $h(i) = f(g(i)) = f(\alpha_i) = \beta_i$. Then the verifier requests h from the prover, receives some function h^* , and checks that $h^*(i)$ does actually equal β_i . We need to check that h^* is actually correct ($h^* = h$). To do this, we sample $\eta \leftarrow \mathbb{F}$ and then compute whether $h^*(\eta) = f(g(\eta))$. So, our protocol only needs to evaluate f once!

- Soundness: suppose that there exists i such that $f(\alpha_i) \neq \beta_i$. If P sends $h^* = h = f \circ g$, V rejects. As we have seen before, if P makes up a false h^* , V rejects with probability $\geq 1 - (n + d)/|\mathbb{F}|$ (and we can make $|\mathbb{F}|$ big enough).
- Completeness: trivial.

Putting it all together

Applying the query reduction protocol to our problem with $f = \hat{V}_{i+1}$, we can reduce the problem of checking that $\eta_1^* = \hat{V}_{i+1}(\gamma_2)$ and $\eta_2^* = \hat{V}_{i+1}(\gamma_3)$ to a single query to the prover, which is

$$\hat{V}_{i+1}(g(\eta_0)) \stackrel{?}{=} h^*(\eta_0),$$

with η_0 uniformly sampled from $\mathbb{F}$ and $h^* : \mathbb{F} \rightarrow \mathbb{F}$ some degree- $\leq 2k$ polynomial sent by the prover. We evaluate $\beta_{i+1} := h^*(\eta_0)$ and $\alpha_{i+1} := g(\eta_0)$ ourselves in $\text{polylog}(n)$ time and use the same procedure as above to check that $\hat{V}_{i+1}(\alpha_{i+1}) = \beta_{i+1}$.

Recurring in this way lets us reduce the problem of checking whether $\widehat{V}_0(00\dots 0) = 1$ to checking $\widehat{V}_d(\alpha_d) = \beta_d$ in time $\text{polylog}(n)$. Can we check the latter efficiently? Recall that

$$V_d(i) = \begin{cases} x_i & \text{if } i \leq n \\ 0 & \text{otherwise,} \end{cases}$$

and

$$\widehat{V}_d(\alpha_d) = \sum_{a \in \{0,1\}^k} V_d(a) \text{EQ}(a, \alpha_d) = \sum_{a=1}^n V_d(a) \text{EQ}(a, \alpha_d),$$

which takes time $n \text{polylog } n$ (if $|\mathbb{F}|$ is polynomial in n).

We also promised the prover could do things efficiently. To evaluate $\widehat{V}_i$, it takes $K \text{polylog } K = \text{poly } n$. To do sumcheck, it takes $O(d2^m)$ for m -variate, degree d sumcheck. Here, $m = 3k$, and $d = 3$, so this is $O(K^3) = \text{poly}(n)$.

September 22, 2026

Lecture 7: Honest-Verifier Statistical Zero-Knowledge

CS 6814: Probabilistic Proofs

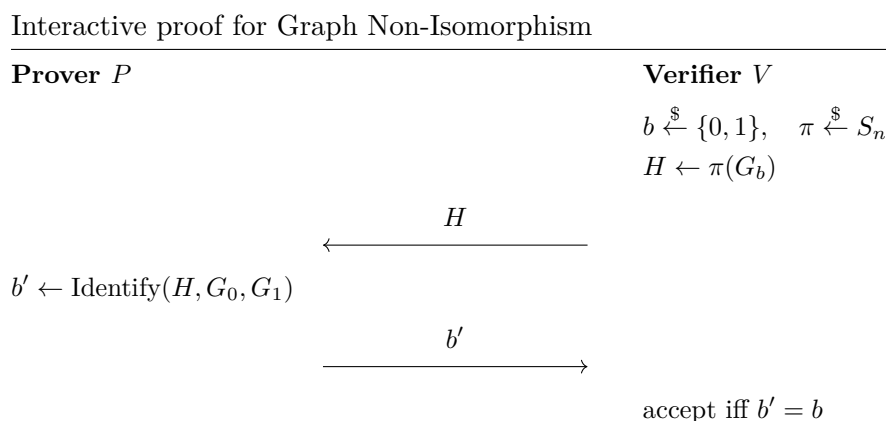
Instructor: Nick Spooner Scribe: Shen Dong

1 Honest-Verifier Statistical Zero-Knowledge

Consider the following protocol for Graph Non-Isomorphism (GNI) for the basic idea of Zero Knowledge proofs. Define the language

$$\text{GNI} := \{(G_0, G_1) : G_0 \not\simeq G_1\}.$$

Suppose the prover and verifier have two non-isomorphic graphs G_0, G_1 , and the statement being proved is $(G_0, G_1) \in \text{GNI}$.



If $G_0 \not\simeq G_1$, the prover can determine which graph H is actually isomorphic to and therefore the verifier accepts with probability 1 (perfect completeness). If $G_0 \simeq G_1$, then the distribution of $\pi(G_b)$ and G_{1-b} are identical, so the message H , as a graph randomly sampled from the distribution, reveals no information about b , so the prover wins with probability at most $\frac{1}{2}$ (soundness). The verifier learns no additional information besides $G_0 \not\simeq G_1$, i.e. learns nothing more than the statement is true. We call this Zero Knowledge.

Why is "learning nothing" important in proof systems? Consider the NP language " $\exists x, \Phi(x) = 1$ ", the prover who claims the statement to be true can provide an x so that the verifier checks it, then verifier knows the statement is true and also has a witness/certificate. However, in many settings of cryptography, the certificate x can be cryptographic secret information, like the secret key x in statements of valid encryption:

$$\{(c, m) : \exists x, \text{Dec}(x, c) = m\},$$

where the prover does not want to reveal x but still wants to convince you the claim. Also Zero Knowledge is interesting in complexity. The class **HVSZK** we are going to define later satisfies

$$\mathbf{HVSZK} \subseteq \mathbf{AM} \cap \mathbf{coAM}.$$

That is, when $L \in \mathbf{HVSZK}$, L is "unlikely" to be **NP**-hard, otherwise we have $\mathbf{NP} \subseteq \mathbf{coAM}$, making the polynomial hierarchy collapsed to the second level.

Definition 1.1 (View of the verifier). For an IP protocol $\langle P, V \rangle$, $\text{view}_V(x)$ is a random variable consisting of V 's random tape and all messages received from P .

Definition 1.2 (Honest-Verifier Statistical Zero-Knowledge). $\langle P, V \rangle$ is honest-verifier statistical ZK (HVSZK) if there exists a PPT simulator S , a negligible function $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ s.t. $\forall x \in L$,

$$\Delta(S(x), \text{view}_V(x)) \leq \varepsilon(|x|),$$

where the Δ is statistical distance (Total Variation Distance).

Definition 1.3. D_0, D_1 are two distributions, their statistical distance is defined as

$$\Delta(D_0, D_1) := \frac{1}{2} \sum_x \left| \Pr_{D_0}[x] - \Pr_{D_1}[x] \right|.$$

Definition 1.4 (Negligible function). A function $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ is called negligible if $\forall c > 0, \exists n_0, \forall n > n_0, \varepsilon(n) \leq n^{-c}$.

For example $\varepsilon(n) = 2^{-n}$ is a negligible function.

Definition 1.5 (HVSZK).

$$\text{HVSZK} := \{L : \exists \text{ an HVSZK IP protocol } \langle P, V \rangle \text{ for } L\}.$$

Proposition 1.6. $\text{GNI} \in \text{HVSZK}$.

Here is the simulator for Protocol 1: $S(G_0, G_1)$ samples $b \xleftarrow{\$} \{0, 1\}$ and $\pi \xleftarrow{\$} S_n$, then outputs $(b, \pi(G_b), b)$ as V 's randomness view and message view. There is zero simulation error because the verifier's view is just its own randomness together with the prover's replay, which equals to b in every Yes instance since the prover is supposed to answer b .

2 Statistical Difference Problem

Definition 2.1.

$$\text{SD}^{\alpha, \beta} := \left(\text{SD}_Y^\alpha, \text{SD}_N^\beta \right).$$

For a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, we define the distribution $D_C \sim C(U_n)$. The yes instance is defined as

$$\text{SD}_Y^\alpha := \{(C_0, C_1) : \Delta(D_{C_0}, D_{C_1}) \geq \alpha\},$$

$$\text{SD}_N^\beta := \{(C_0, C_1) : \Delta(D_{C_0}, D_{C_1}) \leq \beta\},$$

The circuits have the same output length; if not their random-input lengths can be equalized by padding unused input bits. And the input is guaranteed to be either $\Delta \geq \alpha$ or $\Delta \leq \beta$, without falling in between.

We state and prove the following proposition:

Proposition 2.2.

$$\Delta(D_0, D_1) = \Pr_{y \leftarrow D_0} [y \in S] - \Pr_{y \leftarrow D_1} [y \in S],$$

where $S := \{y : \Pr_{D_0}[y] > \Pr_{D_1}[y]\}$.

Proof. We know $\sum_y \Pr_{D_0}[y] = \sum_y \Pr_{D_1}[y] = 1$. Therefore, $\sum_y \Pr_{D_0}[y] - \Pr_{D_1}[y] = 0$, which means that the negative part and the positive part have same mass.

$$\begin{aligned} \frac{1}{2} \sum_y \left| \Pr_{D_0}[y] - \Pr_{D_1}[y] \right| &= \sum_{y \in S} \left(\Pr_{D_0}[y] - \Pr_{D_1}[y] \right) \\ &= \Pr_{y \leftarrow D_0}[y \in S] - \Pr_{y \leftarrow D_1}[y \in S]. \end{aligned}$$

□

Theorem 2.3. For fixed constants $0 \leq \beta < \alpha \leq 1$, with $\alpha^2 > \beta$,

$$\text{SD}^{\alpha, \beta} \in \mathbf{HVSZK}, \text{SD}^{\alpha, \beta} \leq_{\text{poly}} \text{SD}^{1-2^{-n}, 2^{-n}},$$

where n is the input size (size of two circuit descriptions).

First we talk about **polarization** of the statistical differences: getting two distributions D'_0, D'_1 from D_0, D_1 , s.t. when D_0, D_1 are close, then D'_0, D'_1 are very close; when D_0, D_1 are far, then D'_0, D'_1 are very far.

Lemma 2.4 (Direct Product). Define direct product for a distribution D as $D^{\otimes k}$. If $\Delta(D_0, D_1) = \delta$, then

$$1 - 2e^{-k\delta^2/2} \leq \Delta(D_0^{\otimes k}, D_1^{\otimes k}) \leq k\delta.$$

Proof. Proof of the upper bound: Replace the k coordinates one at a time and use the triangle inequality. Each replacement changes the distribution by statistical distance δ .

Proof of the lower bound. Intuitively the lowerbound means that if the original distributions are far apart, then the tensor product will make it very far. This can be obtained with the Chernoff bound. Let $\delta := \Delta(D_0, D_1)$, $S = \{y : \Pr_{D_0}[y] > \Pr_{D_1}[y]\}$, and for simplicity let $p_b := \Pr_{y \leftarrow D_b}[y \in S]$. Then we know $p_0 > p_1$, and by Proposition 2.2 $p_0 - p_1 = \delta$. Given k independent samples $(Y_1, \dots, Y_k) \sim D_b^{\otimes k}$, for all $i \in [k]$ whether Y_i is inside S or not are independent random variables, and they are distributed over a Bernoulli distribution with parameter p_b .

We construct a distinguisher who distinguish the two direct-product distributions by comparing $\frac{1}{k} \sum_{i \in [k]} \mathbf{1}[Y_i \in S]$ with $\frac{1}{2}(p_0 + p_1)$, if the fraction of lying in S is no less than $\frac{1}{2}(p_0 + p_1)$, then the distinguisher answer D_0 otherwise D_1 . Therefore, by the lower tail of Hoeffding bound

$$\Pr_{D_0^{\otimes k}} \left[\frac{1}{k} \sum_{i \in [k]} \mathbf{1}[Y_i \in S] < p_0 - \frac{\delta}{2} \right] \leq e^{-k\delta^2/2}$$

Similarly the upper tail error is also at most $e^{-k\delta^2/2}$, so the distinguishing event has probability gap at least $1 - 2e^{-k\delta^2/2}$. So the statistical distance is also at least $1 - 2e^{-k\delta^2/2}$. □

Lemma 2.5 (XOR Lemma). $D_b^{\oplus k}$ is the distribution of uniformly sampled $b_1, \dots, b_k$, for each i $x_i \leftarrow D_{b_i}$, s.t. $\bigoplus_{i=1}^k b_i = b$. Then output $x_1, \dots, x_k$. Then we have $\Delta(D_0^{\oplus k}, D_1^{\oplus k}) = \Delta(D_0, D_1)^k$.

Proof. For simplicity let p_b be the probability mass function of distribution D_b , i.e. $p_b[x] := \Pr_{D_b}[x]$.

$$\Pr_{D_b^{\oplus k}}[x_1, \dots, x_k] = \frac{1}{2^{k-1}} \sum_{\substack{(b_1, \dots, b_k), \\ \bigoplus_i b_i = b}} (p_{b_1}(x_1) \dots p_{b_k}(x_k))$$

$$\begin{aligned}
\Pr_{D_0^{\oplus k}}[x_1, \dots, x_k] - \Pr_{D_1^{\oplus k}}[x_1, \dots, x_k] &= \frac{1}{2^{k-1}} \sum_{(b_1, \dots, b_k)} (-1)^{b_1 + \dots + b_k} \prod_{i=1}^k p_{b_i}(x_i) \\
&= \frac{1}{2^{k-1}} \prod_{i=1}^k (p_0(x_i) - p_1(x_i)).
\end{aligned}$$

$$\begin{aligned}
\Delta(D_0^{\oplus k}, D_1^{\oplus k}) &= \frac{1}{2} \sum_{(x_1, \dots, x_k)} \left| \Pr_{D_0^{\oplus k}}[x_1, \dots, x_k] - \Pr_{D_1^{\oplus k}}[x_1, \dots, x_k] \right| \\
&= \frac{1}{2^k} \sum_{(x_1, \dots, x_k)} \prod_{i=1}^k |p_0(x_i) - p_1(x_i)| \\
&= \frac{1}{2^k} \sum_{(x_1, \dots, x_k)} |p_0(x_i) - p_1(x_i)|^k \\
&= \Delta(D_0, D_1)^k.
\end{aligned}$$

□

Basically we can make the polarization $\Delta(D_0, D_1)^k$ as small as we want (remember we hope 2^{-n}) as long as the exponent k is $\text{poly}(n)$, but the circuit size grows with that.

Claim 2.6. $\text{SD}^{1-2^{-n}, 2^{-n}} \in \mathbf{HVSZK}$.

Interactive proof for Statistical Difference

Prover P

Verifier V

$b \xleftarrow{\$} \{0, 1\}, \quad r \xleftarrow{\$} \{0, 1\}^n$
 $y := C_b(r)$

$\xleftarrow{y}$

$b' := \max_x \Pr[C_{b'}(x) = y]$

$\xrightarrow{b'}$

accept iff $b' = b$

Completeness: if $\Delta(D_0, D_1) \geq 1 - 2^{-n}$,

$$\begin{aligned}
\Pr[b \neq b'] &= \frac{1}{2} \left(\Pr_{y \leftarrow D_0}[y \notin S] + \Pr_{y \leftarrow D_1}[y \in S] \right) \\
&= \frac{1}{2} \left(1 - \Pr_{y \leftarrow D_0}[y \in S] + \Pr_{y \leftarrow D_1}[y \in S] \right) \\
&= \frac{1}{2} (1 - \Delta(D_0, D_1)) \\
&\leq 2^{-n}
\end{aligned}$$

Soundness: If $\Delta(D_0, D_1) \leq 2^{-n}$, then for an arbitrary P^* , with the fact difference of the two conditional probabilities is at most statistical distance,

$$\begin{aligned}
\Pr[b = b'] &= \frac{1}{2} (\Pr[b' = 0 \mid b = 0] + 1 - \Pr[b' = 0 \mid b = 1]) \\
&\leq \frac{1}{2} (1 + \Delta(D_0, D_1)) \\
&\leq \frac{1}{2} + 2^{-n}.
\end{aligned}$$

HVSZK The simulator is statistical zero knowledge.

The PPT simulator samples $b \leftarrow \{0, 1\}$ and $r \leftarrow \{0, 1\}^n$, and outputs (b, r, b) , i.e. similar to the GNI protocol, it uses the sampled challenge bit as the simulated prover's reply. The real view is $(b, r, P(C_b(r)))$. The two views use the same randomly sampled (b, r) , and differ only when the honest prover guesses incorrectly. Hence, on every yes instance:

$$\Delta(S(C_0, C_1), \text{view}_V(C_0, C_1)) \leq \Pr[b \neq b'] = \frac{1 - \Delta(D_0, D_1)}{2} \leq 2^{-n-1}.$$

September 24, 2026

CS 6814 Scribe Notes 9/24

Alexander He

1 Introduction

Last lecture, we completed the proof that the statistical difference (SD) problem is honest-verifier statistical zero knowledge (HVSZK) complete.

Definition: a proof system (P, V) is HVSZK if there exists a probabilistic polynomial time (PPT) simulator S and negligible function $\varepsilon : \mathbb{N} \rightarrow \mathbb{N}$ such that

$$\forall x \in \mathcal{L}, \Delta(S(x), \text{view}_V(x)) \leq \varepsilon(|x|). \quad (1)$$

The set HVSZK is the set of languages $\mathcal{L}$ such that $\mathcal{L}$ has an HVSZK IP. We showed that the graph non-isomorphism problem $(\text{GNI}) \in \text{HVSZK}$ because a simulation of the interactive proof only outputs $S \rightarrow (b, \pi, b)$.

For two distributions D_0, D_1 on the same support $\mathcal{X}$, their statistical difference is defined as

$$\Delta(D_0, D_1) = \frac{1}{2} \sum_{x \in \mathcal{X}} |p_{D_0}(x) - p_{D_1}(x)|, \quad (2)$$

where $p_D(x)$ denotes the probability that distribution D assigns to outcome x .

A binary circuit is a map $C : [0, 1]^n \rightarrow [0, 1]^m$ from n bits to m bits. We can define a distribution from the circuit $D_c \equiv C(\mathcal{U}_n)$, where $\mathcal{U}_n$ is the uniform distribution on n bits.

The Statistical Difference problem $\text{SD}^{\alpha, \beta}$ is parameterized on $\alpha, \beta \in [0, 1]$ and consists of two sets:

$$\text{SD}_Y^\alpha = [(C_0, C_1) : \Delta(D_{C_0}, D_{C_1}) \geq \alpha] \quad (3)$$

$$\text{SD}_N^\beta = [(C_0, C_1) : \Delta(D_{C_0}, D_{C_1}) \leq \beta] \quad (4)$$

Given two circuits C_0, C_1 , it must be determined which set the pair belongs to.

We will prove the following theorem: for all α, β such that $\alpha^2 \geq \beta$, $\text{SD}^{\alpha, \beta}$ is complete for HVSZK. i.e. for all $\mathcal{L} \in \text{HVSZK}$, $\mathcal{L} \leq_p \text{SD}^{\alpha, \beta}$.

Last time, we showed that for arbitrary α, β , $\text{SD}^{\alpha, \beta}$ can be reduced in polynomial time to $\text{SD}^{1-2^{-n}, 2^{-n}}$.

Proof continued:

Suppose $\mathcal{L}$ has an HVSZK proof which is public coin and 2-message (AM). We will show that $\mathcal{L}$ can be reduced to SD. The proof can be extended to languages that are private coin and any number of messages [Sahai-Vadhan '03]. In general, the protocol for $\mathcal{L}$ consists of the verifier sending $r \leftarrow [0, 1]^R$ and the prover sending back a message m . The verifier will compute $V(r, m) \rightarrow \{0, 1\}$ and either accept or reject. Assume that the protocol's completeness is $1 - 2^{-n}$ and its soundness is 2^{-n} . Let S be the simulator.

To reduce this to SD, we want some relevant statistical difference to be small when $x \in \mathcal{L}$ and the difference to be large when $x \notin \mathcal{L}$. In particular, the reduction should perform

$$x \in \mathcal{L} \rightarrow (C_0, C_1) \text{ such that } D_0 \approx_\varepsilon D_1 \quad (5)$$

$$x \notin \mathcal{L} \rightarrow (C_0, C_1) \text{ such that } \Delta(D_0, D_1) \approx \frac{1}{2} \quad (6)$$

If $x \in \mathcal{L}$, then $S(x) \approx_\varepsilon \text{view}_V(x) = (r, P(r))$, meaning the output of the simulator will be close randomness and the prover response. Running V on the output of the simulator gives $\Pr_{r \leftarrow [0,1]^n} [V(S(x)) = 1] \geq 1 - 2^{-n} - \varepsilon(n)$.

If $x \notin \mathcal{L}$, then $\Pr[\exists m. V(r, m) = 1] \leq 2^{-n}$ by soundness. In this case, we can still compute $V(S(x))$. If it outputs yes, then it means the simulator generated distribution doesn't look random. If it's no with large probability, then it means that $\mathcal{L} \in \text{BPP}$. $V(S(x))$ in general can be computed in PPT time. Thus, we define the following distributions:

$$D_0 = \text{run } S(x) \rightarrow (r, m), \text{ output } (r, V(r, m)) \quad (7)$$

$$D_1 : r \leftarrow [0, 1]^R, \text{ output } (r, 1) \quad (8)$$

If $x \in \mathcal{L}$, by soundness and completeness, $D_0 \approx_\varepsilon D_1$. Then $\Delta(D_0, D_1) \leq 2^{-n} + 2\varepsilon(n) = \text{negl}$. If $x \notin \mathcal{L}$, then $\Delta(D_0, D_1) \geq \max[\Delta(r \rightarrow S, \mathcal{U}), \Delta(V(r, m), 1)]$, where $(r, m) \leftarrow S$. Then

$$\Pr_{(r,m) \leftarrow S} (V(r, m) = 0) \geq \Pr_{r \leftarrow S} (\forall m, V(r, m) = 0) \quad (9)$$

$$\geq \Pr_{r \leftarrow \mathcal{U}} (\forall m, V(r, m) = 0) - \Delta(r \leftarrow S, \mathcal{U}) \quad (10)$$

$$\geq 1 - 2^{-n} - \Delta(r \leftarrow S, \mathcal{U}) \quad (11)$$

The first probability is the probability that $V(r, m) = 0$ for some particular m , so it is lower bounded by the probability that $V(r, m) = 0$ for all m . The second line comes from the negation of the soundness condition, the triangle inequality, and the fact that the probability of an event under two different distributions can only change by a maximum of the statistical difference between the two distributions. Thus, $\Delta(D_0, D_1) \geq \max(\delta, 1 - 2^{-n} - \delta) \geq \frac{1-2^{-n}}{2}$, where $\delta = \Delta(r \leftarrow S, \mathcal{U})$. Thus, for some negligible function ε' ,

$$x \in \mathcal{L} \implies \Delta(D_0, D_1) \leq \varepsilon'(n) \quad (12)$$

$$x \notin \mathcal{L} \implies \Delta(D_0, D_1) \geq \frac{1}{2} - \varepsilon'(n). \quad (13)$$

This is a reduction from $\bar{\mathcal{L}} \leq_p \text{SD}^{\frac{1}{2}-\epsilon, \epsilon} \leq_q \text{SD}^{\alpha, \beta}$, which means that $\mathcal{L} \leq_p \overline{\text{SD}^{\alpha, \beta}}$. Thus, $\text{SD}^{\alpha, \beta}$ is HVSZK complete.

Caveat: we have shown that for all $\mathcal{L} \in \text{HVSZK}$, $\bar{\mathcal{L}} \leq_p \text{SD}$, or that $\mathcal{L} \leq_p \overline{\text{SD}}$. However, we want to show that $\mathcal{L} \leq_p \text{SD}$. We know that $\text{SD}^{\alpha, \beta} \in \text{HVSZK}$, and that $\overline{\text{SD}} \leq_p \text{SD}$ by completeness. Similarly, $\text{SD} \leq_p \overline{\text{SD}}$. Thus, $\bar{\mathcal{L}} \leq_p \overline{\text{SD}} \iff \mathcal{L} \leq_p \text{SD}$.

Consequences:

- HVSZK = co-HVSZK: SD is complete for HVSZK and co-SD is complete for co-HVSZK. Thus, $\text{SD} = \text{co-SD} \implies \text{HVSZK} = \text{co-HVSZK}$.
- $\text{HVSZK} \subseteq \text{AM} \cap \text{co-AM}$:

$$\begin{aligned}
\text{SD} &\subseteq \text{HVSZK}(2) \\
&\subseteq \text{IP}(2) \\
&\subseteq \text{AM}(4) \\
&\subseteq \text{AM}
\end{aligned}$$

This means $\overline{\text{SD}} \subseteq \text{co-AM}$. If $\text{NP} \subseteq \text{co-AM}$, the polynomial hierarchy collapses. Similarly, if $\text{NP} \subseteq \text{HVSZK}$, then the polynomial hierarchy collapses. Thus, problems with HVSZK proofs are probably not NP-hard.

2 General Zero-Knowledge

Definition: (P, V) is (malicious verifier) SZK if:

$$\forall \text{PPT } V^*, \exists \text{PPT } S_{V^*}, \text{negl. } \varepsilon \text{ s.t. } \forall x \in \mathcal{L}, \Delta(S_{V^*}(x), \text{view}_{V^*}(x)) \leq \varepsilon(|x|). \quad (14)$$

For example, for the GNI IP, we can show that (P, V) is not SZK unless $\text{GI} \in \text{BPP}$. Equivalently, we can show that if (P, V) is SZK, then $\text{GI} \in \text{BPP}$. (This will be a homework problem)

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [GS86] S Goldwasser and M Sipser. Private coins versus public coins in interactive proof systems. In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing, STOC '86*, page 59–68, New York, NY, USA, 1986. Association for Computing Machinery.
- [JG79] David S Johnson and Michael R Garey. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman, 1979.
- [Sha92] Adi Shamir. $\text{Ip} = \text{pspace}$. *Journal of the ACM (JACM)*, 39(4):869–877, 1992.